

Design Principles

The Codynamic Book Machine is built around a small set of design principles that keep authoring work inspectable, recoverable, and easy to coordinate across agents. First, state is durable rather than ephemeral: outlines, section payloads, task queues, logs, and generated artifacts are written to repository-backed files so that the system can resume from recorded history instead of relying on hidden conversational memory. This makes the manuscript itself a persistent object, not a transient chat outcome.

Second, work is routed through explicit queues and visible messages. Agents do not silently mutate shared state; they receive declared tasks, choose an action that matches the task, and record their work as callbacks or append-only messages. That structure makes coordination legible to both humans and other agents, because each handoff can be inspected after the fact. It also reduces ambiguity about what was requested, what was completed, and what remains pending.

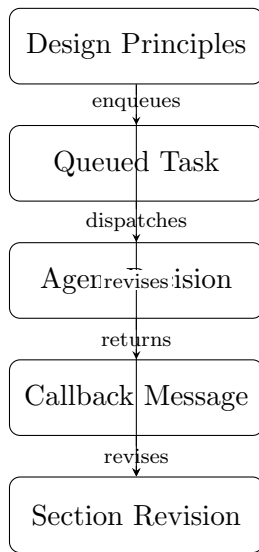
Third, the architecture assigns specialist ownership to distinct kinds of work. Section agents draft and revise section payloads, the gardener performs editorial maintenance, the references agent owns the shared bibliography, and the diagram agent manages visual assets and their memory. This division of labor keeps each agent focused on a narrow responsibility and prevents ad hoc edits from bypassing the system’s checks. When a section needs support that it does not own, the request is routed to the appropriate specialist rather than improvised locally.

Fourth, edits are intended to be reversible and reviewable. Section payloads are stored separately from imported source material, so a draft can be regenerated, compared, or repaired without destroying the original notes. The same principle applies to revisions triggered by gardening or compilation feedback: the system prefers targeted changes over opaque rewrites, which makes it easier to understand why a line changed and to roll back if necessary.

Fifth, the system favors visible messages over hidden side effects. Inter-agent communication is represented as durable task records and readable chat lines, which means the coordination process itself becomes part of the manuscript’s operational evidence. This visibility is important for debugging, editorial review, and later explanation of how a given section came to be.

Finally, the authoring workflow is schema-first. The canonical work object, its recursive outline, and the section payload model define the shape of the project before prose generation begins. By normalizing intent into structured fields and section-level responsibilities, the system can reason about dependencies, citations, diagrams, and compilation as parts of one coherent object. In practice, that means the prose is not merely written into a repository; it is authored within a machine-readable contract that keeps content, process, and artifacts aligned.

Taken together, these principles make the system less like a single-pass text generator and more like a durable editorial machine. The manuscript can be inspected at every layer, from outline to queue to TeX to compiled artifact, while each agent remains accountable for a specific slice of the work.



Conceptual diagram for 'Design Principles': Summarize the architecture's principles: durable state, explicit queues, specialist ownership, reversible edits, visible messages, and schema-first authoring.

Figure 1: Conceptual diagram for “Design Principles”: Summarize the architecture’s principles: durable state, explicit queues, specialist ownership, reversible edits, visible messages, and schema-first authoring.